

**Centro de Enseñanza Técnica y Superior**



Lenguajes de programación orientada a objetos  
Dra. Vanessa Miranda Lopez

Proyecto final: 4 Amigos

Presentado por:

Sahid Eleazar Velazquez Murillo - T041528  
Aarón Avalos Salinas - T042205  
Haniel Alejandro Acosta Masias - T040816  
Fernando Antonio Zavalza Verdugo - T040198

Ingeniería en Ciencias Computacionales.

Tijuana, B.C. a 01 de Junio del 2026

## Reporte técnico

El proyecto trata de imitar a la perfección el juego de mesa conocido como “Cuatro Amigos” un juego que consta de cuatro jugadores utilizando la baraja inglesa y jugando con reglas similares a las del juego “UNO” con pequeñas diferencias en su jugabilidad.

### 1. Pruebas.

- a. Módulo para voltear cartas: Aquí se creó una función para voltear la carta, para ello, se imprime un mensaje “se volteo la carta” como son muchas cartas, el mensaje se repite varias veces.
- b. Inicializar la mano: Con el constructor de jugador y heredada para bot, llama a una función miembro privada llamado inicializar miembro, en ese caso se decidió hacer un cout donde se imprima la frase “Se inicializó la carta”
- c. Recibir las cartas de la mesa a Luigui: La función recibir baraja sirva para recibir una vector bidimensional y se valida con un cout indicando que esas cartas pasaron al vector bidimensional de Luigui.

## Manual técnico:

- Utilizar el programa:
  - 1. Entrar al repositorio de github
  - Descargar la carpeta 4 amigos
  - Entrar a la carpeta descargada
  - Entrar a la carpeta llamada “x64/Debug”
  - Ejecutar el archivo llamado “4amigos.exe”

- Tutorial de uso:

1. Se despliega la pantalla de inicio
2. Haz clic izquierdo sobre el botón de “Play”
3. ¿Qué identificar? Al iniciar la partida tienes tres manos
  - a. La mano principal es la mano que está abajo de la mano final: Es la mano jugable.
  - b. La mano de la reserva son las cartas arriba de la mano final: Después de cierta condición, al cumplirse y darle click entonces esas cartas pasan a la mano principal.
  - c. La mano final son las únicas tres cartas volteadas juntas: Después de cierta condición, al cumplirse y darle click entonces esas cartas pasan a la mano principal.
  - d. ¿Qué condiciones hay que hacer para jugar la mano final y la reserva?
    1. Para la mano reserva, necesita haberse acabado el buche de cartas y las cartas de tu mano principal para agarrar una.
    2. Para la mano reserva, necesitar acabar la mano de la reserva y la mano principal.
  - e. ¿Qué hay en la mesa aparte de mis cartas?
    1. En cada lado de la mesa hay jugadores que compiten por ser el primero en acabar todas sus cartas.
    2. En el centro están las cartas del buche y las cartas comunitarias. No pertenecen a un jugador.
    3. El buche es el principal mecanismo del juego.

4. Automáticamente, comerás de las cartas comunitarias hasta un máximo de tres mientras que haya cartas ahí.

f. ¿Cómo hago para sacar las cartas?

Revisa el buche de cartas. Haz clic a la carta con el mismo valor y un valor más grande o los comodines.

G. ¿Qué pasa si no tengo cartas para poner en el buche?

Tendrás que comerte el buche y saltar tu turno.

H. ¿Cuáles son los comodines?

1. Las cartas con el número 10: Puedes lanzarla no importando el valor de la carta del buche, a cambio, eliminarás todas las cartas del buche para siempre.
2. Las cartas con el número 2: Puedes lanzarla no importando el valor de la carta del buche, a cambio, te dará un turno extra para jugar.

J. Mecánicas extras:

Si en el buche, hay cuatro cartas seguidas del mismo valor, entonces se eliminan las cartas.

Puedes lanzar múltiples cartas siem

K. Valores de los símbolos:

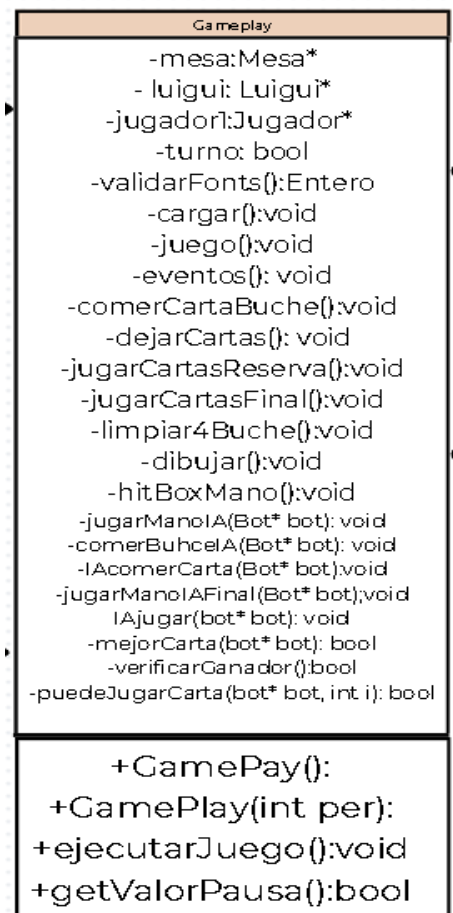
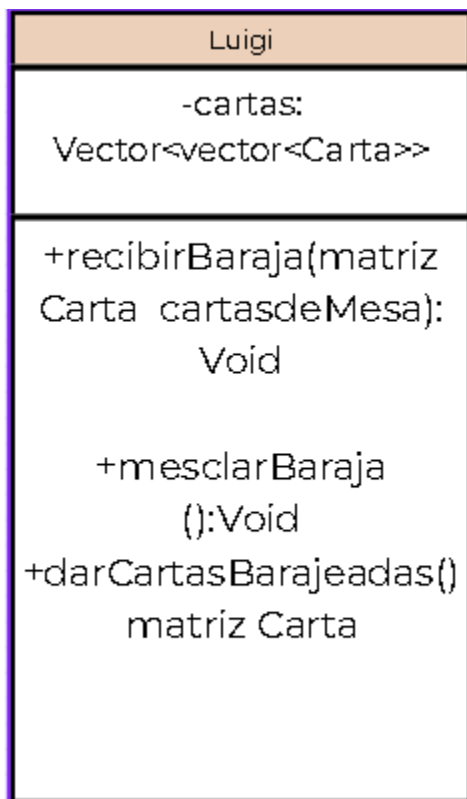
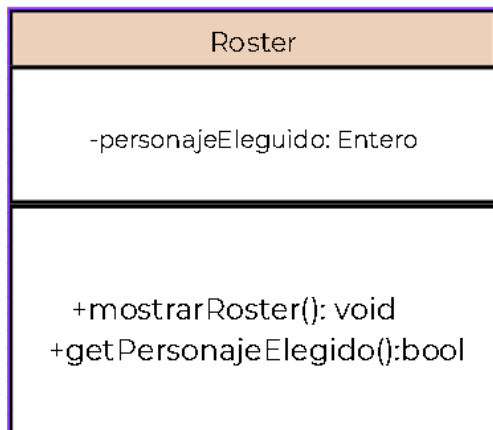
“J” = representa el número once

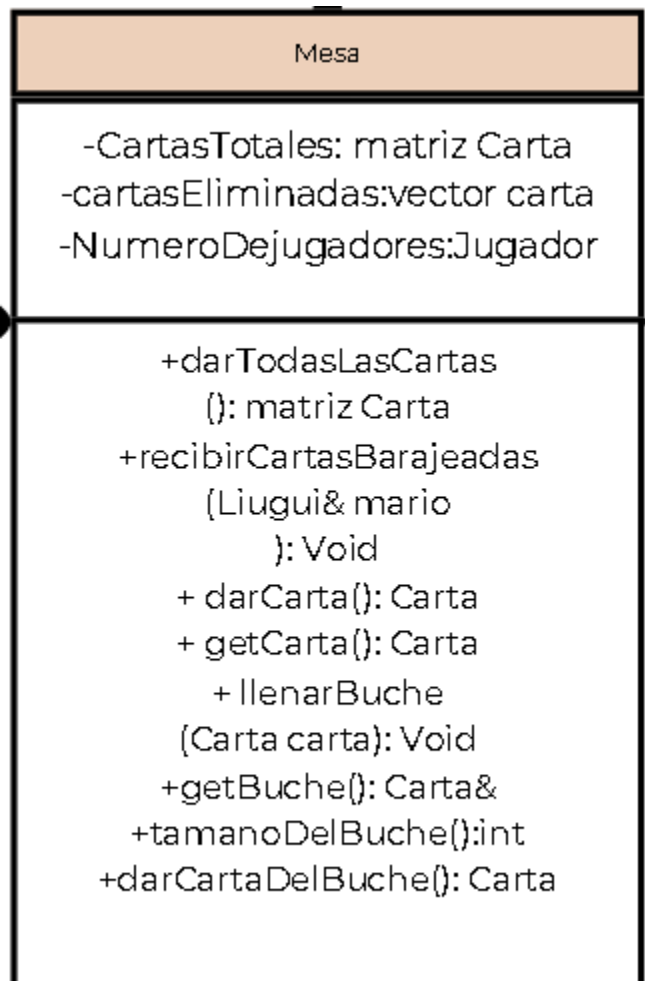
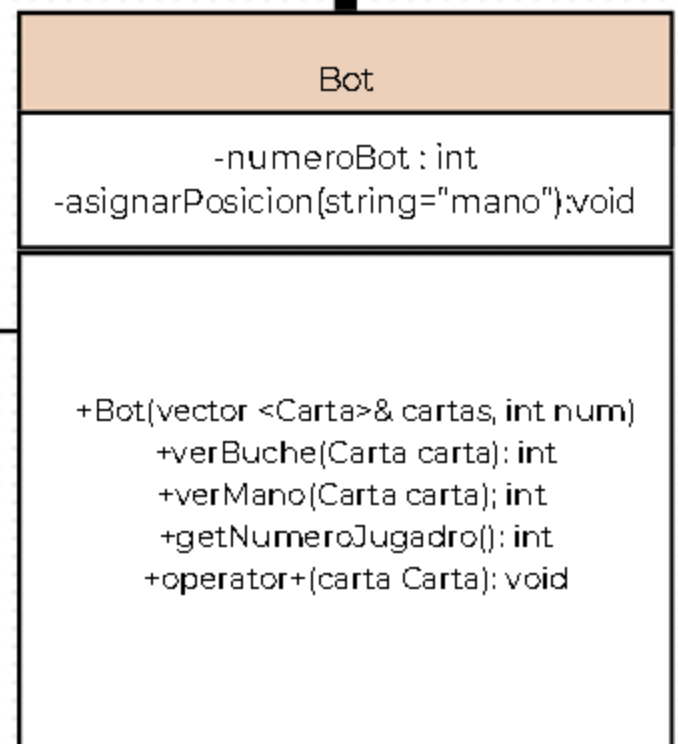
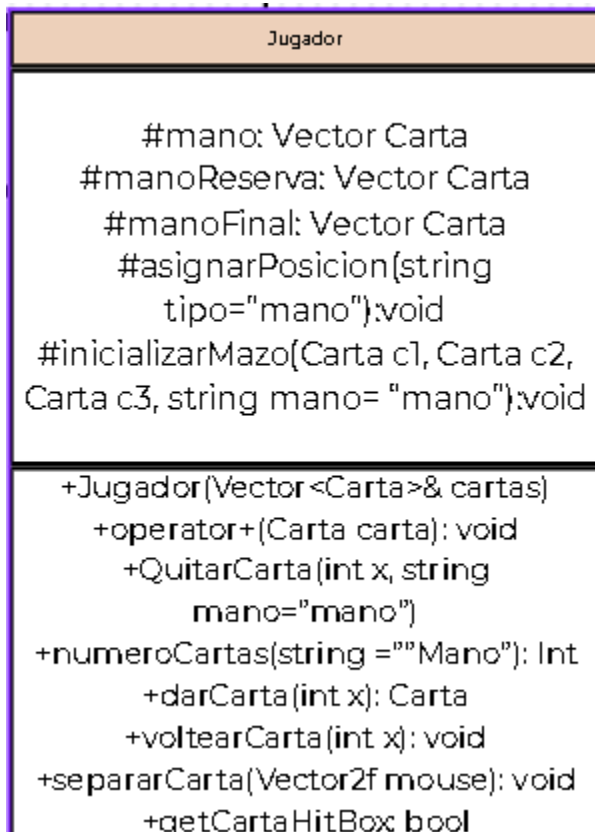
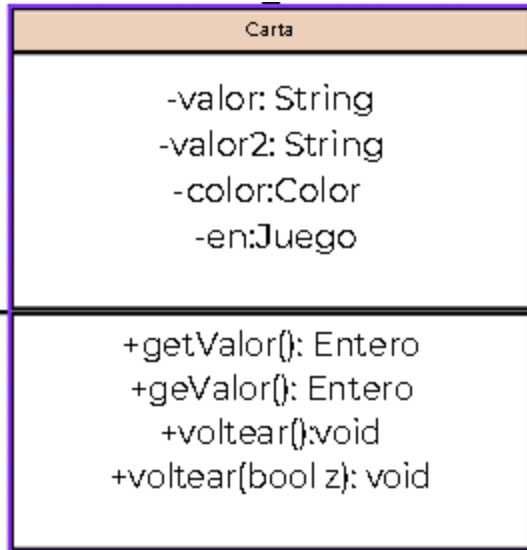
“Q” = representa el número doce

“K ” = representa el número trece

“A” = representa el número catorce

## Diagramas UML





Se lista los tipos de relaciones:

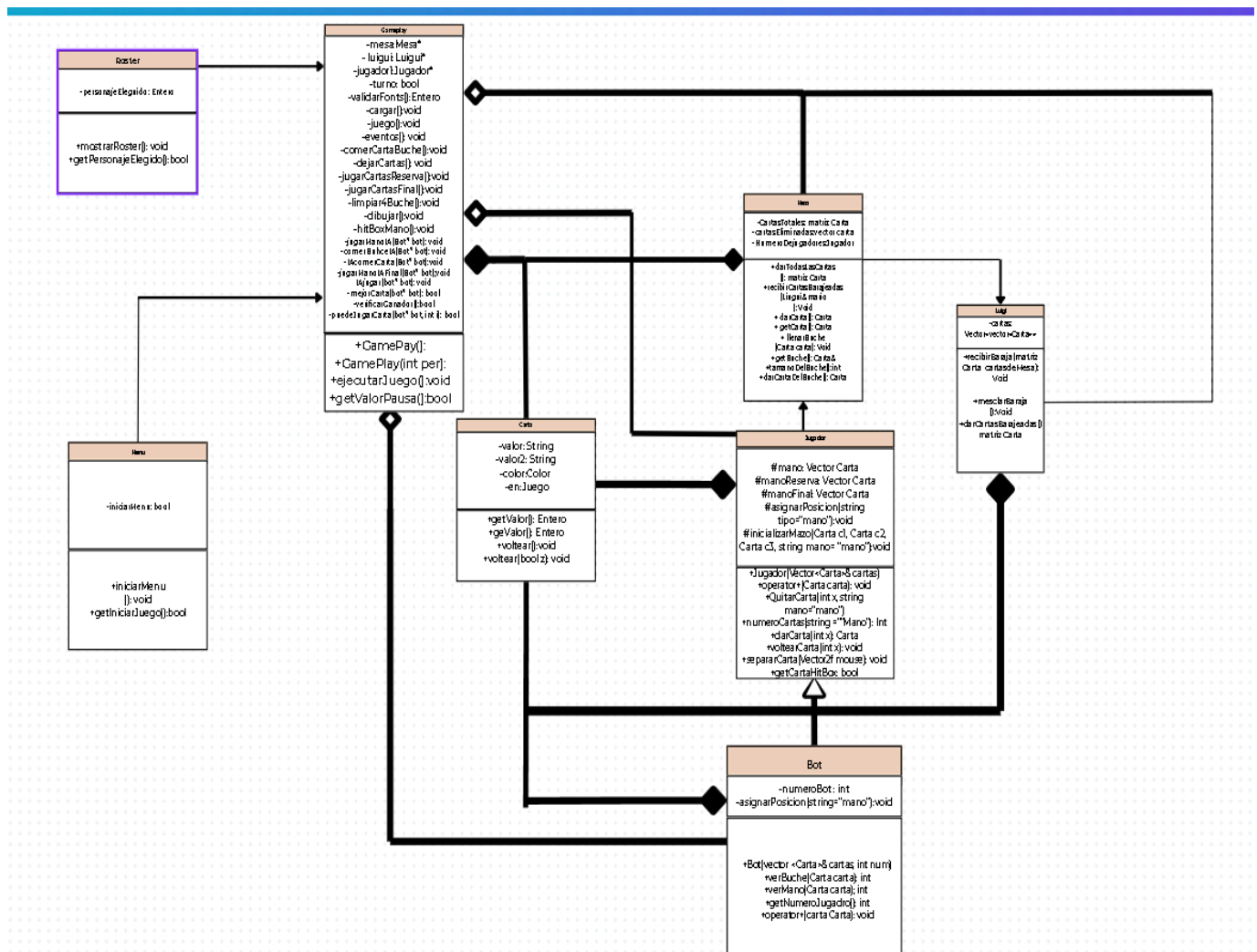
Para la asociación: Las clases Roster y Menú se relacionan con Gameplay por medio de la asociación. También Mesa se relaciona con Jugador y Bot por medio de relación para compartir cartas y Mesa con Luigui comparten la misma relación.

Para Agregación: La clase Gameplay recibe una relación de agregación con las siguientes clases: Mesa, Jugador, Bot, Gameplay, Luigui.

Para la composición: Todas las clases Mesa, Jugador, Gameplay, Bot y Luigui se componen de objetos tipo Carta.

Para herencia: Esa clase Bot heredia de forma pública a la clase Jugador.

Como se puede Observar la imagen, así se relacionan las clases entre sí.

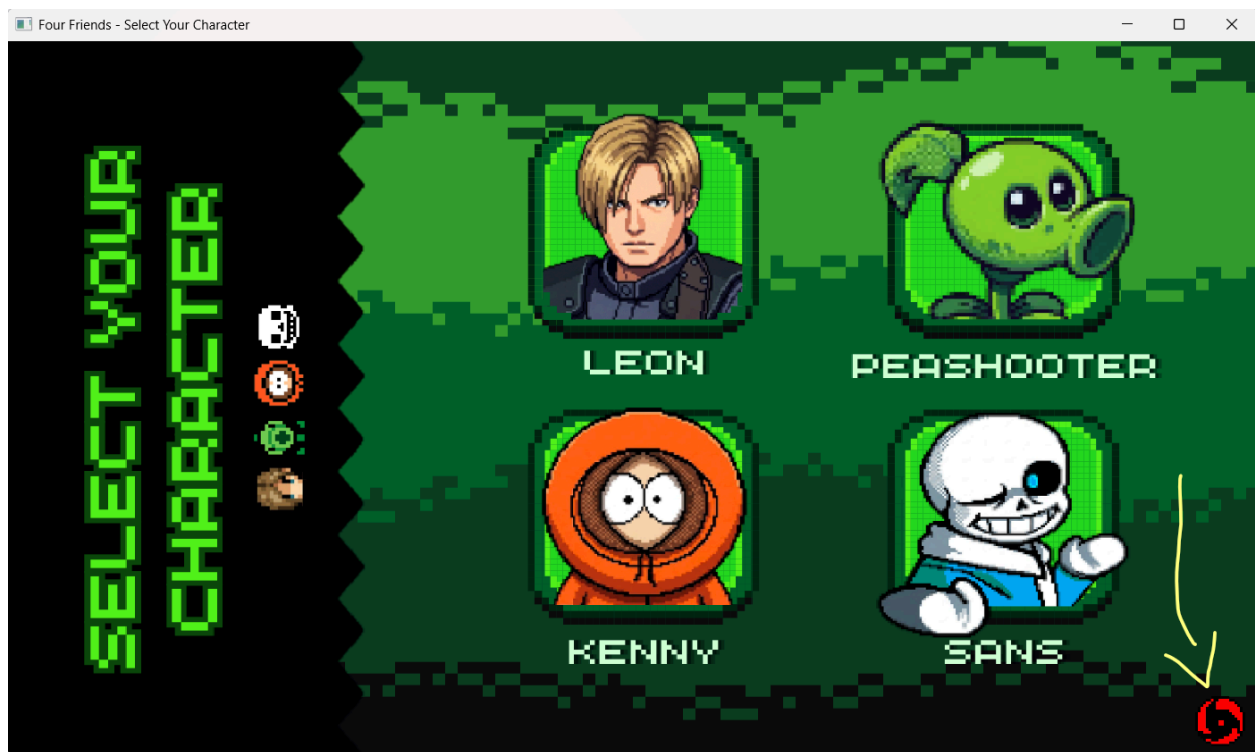
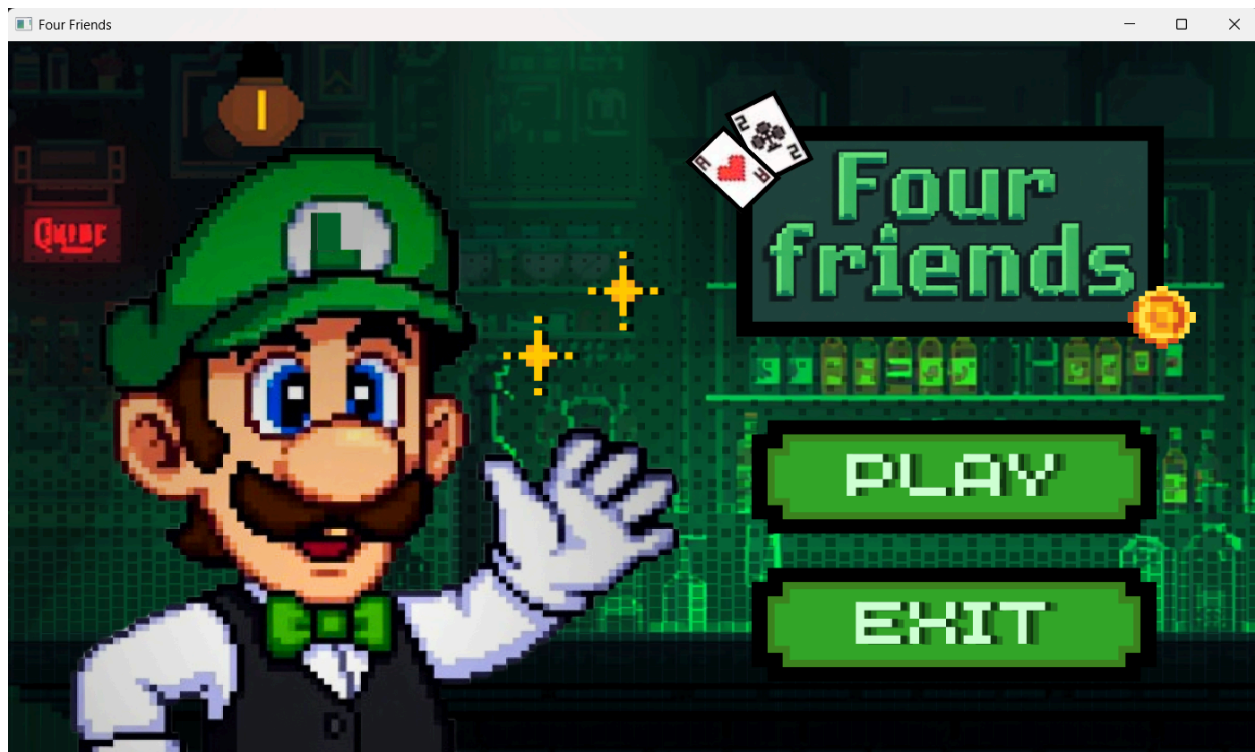


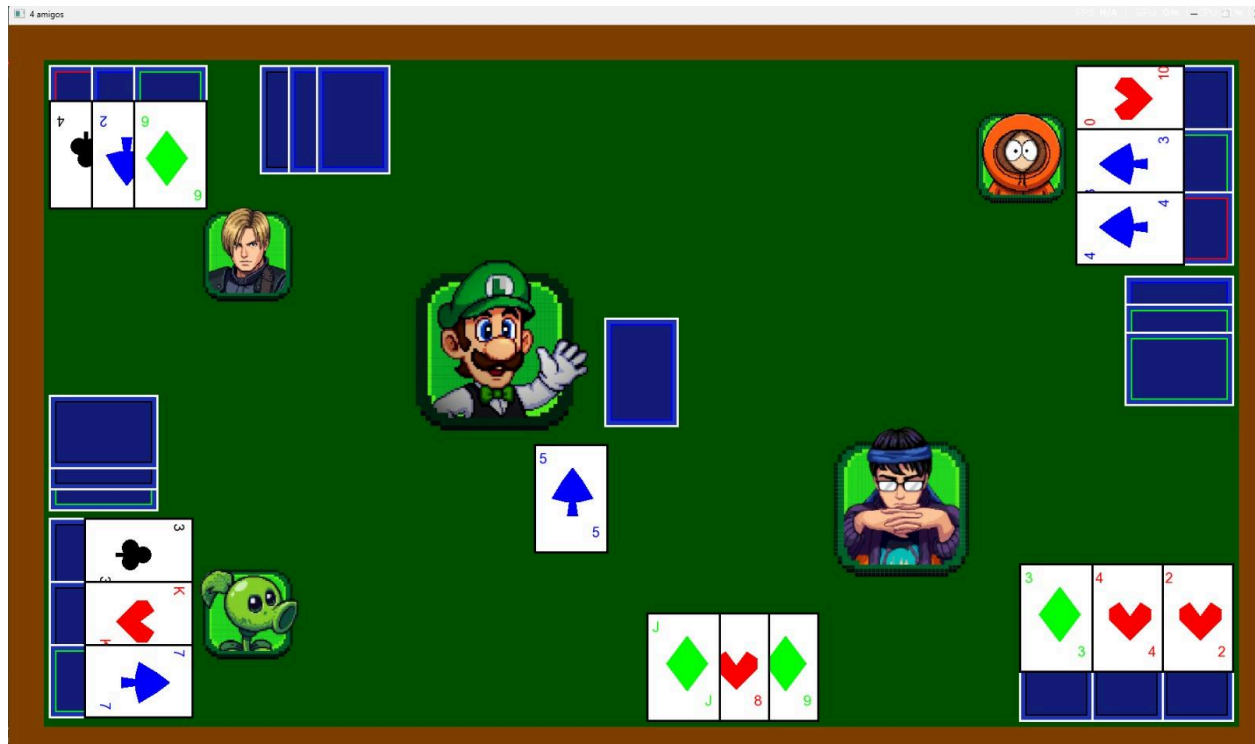
## Justificación de conceptos de POO

| Concepto              | Como se aplico                                                                                                                                                                                                                                                                                   |
|-----------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Abstracción           | Se incorporó a cada clase los atributos necesarios para el proceso, se separan las tareas para cada clase y se muestra al usuario lo únicamente necesario.                                                                                                                                       |
| Encapsulamiento       | Cada clase tiene sus funciones y atributos privados para que el usuario no tenga acceso a cambiar algo del objeto que puede ser contraproducente. El main y entre clases pueden acceder a funciones públicas que solo se necesitan para el flujo del juego, son las famosas funciones set y get. |
| Poliformismo estatico | Muchas funciones de la clase Jugador fueron simplificadas gracias a las funciones sobrecargadas y con parámetros predeterminados                                                                                                                                                                 |
| Poliformismo dinamico | Las funciones void draw() y el operador sobrecargado de jugador fueron sobrescritos.                                                                                                                                                                                                             |
| STL                   | Se utilizaron mucho los vectores, principalmente para guardar objetos cartas, también para hacer algunas validaciones y guardar objetos dinámicos en la clase GamePlay                                                                                                                           |
| Herencia              | La clase Bot es hija de la clase Jugador porque se quiso que las funciones públicas fueran mantenidas para la clase Bot                                                                                                                                                                          |



Capturas de pantalla:





### Conclusiones:

El programa cumple con los pilares orientados a objetos, existe y fue de mucho uso las relaciones entre objetos, así como una relación de herencia para no empezar una clase desde cero. El programa no se rompe, aún persisten algún bug visual por dar muchos clicks seguidos pero en general son bug visuales que duran microsegundos. Lamentablemente no se pudo incluir el multijugador ni el tipo de dificultades para cada Bot por cuestiones de tiempo y complejidad en la ejecución.

### Conclusiones individuales:

#### Sahid:

1. Que aportó al proyecto.

Las mecánicas implementadas para la inteligencia artificial del juego e implementarlas en una clase y desarrollarla completa, así como parte del diseño de la interfaz de usuario como fue el acomodo de los tipos de mano del jugador y las IAs, también algunas funciones de lógica del

jugador como la de separar carta de la mano principal para diferenciar cual carta es la que está por seleccionar. También fui el encargado de resolver dudas de Github a mis compañeros de equipo. También me encargue de mejorar las clases Carta, Gameplay y muy poco de la clase Mesa (junto a mi compañero Aarón).

## 2. Qué retos técnicos enfrentó.

Aparte de los retos que fue el uso de una herramienta como Github sin el conocimiento previo, fue todo un reto, pues no entendía a qué le movía sin embargo me planteé estudiar y al final termine convirtiéndome en el asesor de Github de mi equipo. Además de eso, también me tope con bastantes retos con el desarrollo de la inteligencia artificial, pues el hacer que la máquina discerniera entre las cartas para saber qué carta es la mejor opción me llevo mucho tiempo además de todos los bugs que surgían, yo creo que esos fueron los problemas o retos más fuertes que enfrente en este proyecto (en lo personal).

## 3. Qué habilidades personales o de equipo mejoró.

Pienso que mejore el área del desarrollo lógico de mis habilidades cognitivas, pues al trabajar directamente con la IA, tenía que pensar mucho en cómo hacer que la misma IA tomará decisiones (lo que requería mucha lógica). Otra área que creo yo que mejore en mi persona en el trabajo colaborativo fue el “aprender a enseñar”, con esto quiero dar a entender que me desarrolle mejor en el área de expresarme mejor para que las indicaciones dadas sean entendidas.

### **Fernando (Heefler1945):**

¿Qué aportó al proyecto?

Mi aportación fue en la creación de elementos visuales, como el menú principal, la pantalla de selección de personajes, los personajes e implementarlos en medio del juego para que cada jugador en el juego tenga su respectivo personaje, y entre otros con el fin de darle su propia esencia. También aporté en pulir la interacción con el menú principal y sobre todo la creación

de la pantalla de selección de personajes, con todo y botones para interactuar, sin mencionar el añadido de personajes secretos.

¿Qué retos técnicos enfrentó?

Mi mayor problema fuera del código fue el de adaptarme a GitHub mediante la terminal de Visual Studio Code, esto se debía más que nada a la falta de experiencia pero eventualmente pude adaptarme. Otro problema que me enfrenté fue el de implementar los assets de la pantalla principal y de selección de personajes, porque gran parte de su desarrollo consistía en hacer prueba y error hasta que milagrosamente me salió la pantalla del menú junto con el asset que había creado, y con eso pude hacer la pantalla de selección con facilidad utilizando el código del menú como base. Sin embargo, todavía faltaba acomodar los botones e íconos de personajes (en la partida) en sus respectivas posiciones y para hacer eso era completamente prueba y error.

¿Qué habilidades personales o de equipo mejoró?

Pude mejorar mi conocimiento en la programación, entender mucho mejor como funciona todo lo que he visto a lo largo de la materia y sobre todo tener una idea de lo que representa la creación de un videojuego. En cuanto a habilidades de equipo, aprendí a no tener miedo de pedir ayuda y siempre tener presencia e involucramiento en el equipo. Y que hasta cierto punto, el desarrollo de un programa se pone mucho más divertido cuando lo entiendes y sabes que vas a buen camino.

**Haniel (Lesenblink):**

Mis mayores aportes diría que fueron la creación del menú y su funcionamiento, la creación del menú de salida (a excepción del botón para regresar al menú principal), parte del cambio del sistema de arrastre hacia el de click. Los mayores problemas que tuve fueron el uso de la

plataforma Github (mediante la consola de Visual Studio), la permanencia de la ventana del menú después de ganar o perder y la estructura con esta. Creo que mi percepción de los problemas mejoró, además de aceptar al buscar ayuda sin remordimientos y note una mejora en mi estructura sobre la lógica de programación.

El mayor problema con el que me encontré dentro del código fue la aplicación de la pantalla final. Esta se cerraba inmediatamente a la hora de que se llegaba a crear, lo cual era algo bastante fácil de notar. En la búsqueda de encontrar el resultado, estuve intentando forzar la solución en otro aspecto del programa cuando realmente había una falla simple en el código, haciendo el problema más grande de lo que debía ser. A pesar de probar diferentes alternativas, tanto planeadas como ya hechas a la suerte para encontrar una solución, nada funcionaba.

Así que me vi forzado a tomar un paso atrás y mirar todos los aspectos del código, revisar cada aspecto y encontrar fallas que no me puse a considerar. Ahí fue cuando encontré la falla. Un pedazo de código crucial para la creación de ventanas estaba faltando, cosa que forzaba la ventana a cerrarse. Otro problema que tuve fue buscar la lógica para decidir el texto de victoria y derrota. Cosa que estuve sobrepensando alternativas de aplicarla de una forma simple, cuando lo único que hacía era sobre complicar la respuesta. Cosa que rápidamente cambió cuando recordé una interacción entre funciones que me permitió crear la condición sin necesidad de una necesitar cosas fuera de las funciones ya establecidas para su uso.

La relación en la que esta función tendría con el resto de las otras funciones, al estar todas dentro de una misma clase, era necesario que esta función no interfiriera con aspectos no deseados. Aunque esta función a la hora de crearse era mayormente contenida en funciones específicas, sigue siendo necesario el considerar como ésta actúa a la hora de que el programa esté en acción sin limitarse a que funcione, pero el que su función no intervenga con el resto del programa de una forma indeseada.

Considero que es un motivo de ingeniería en general debido a la naturaleza de estas. No son simples problemas de compilación, sino una falla en la lógica del mismo problema, aspecto que va más allá de solo un problema de compilación, más que necesita un enfoque en cómo una persona enfrenta un problema y cómo encuentra la solución, teniendo que formar un pensamiento estructurado y calmado para encontrar la solución en lugar de solamente cambiar variable a fuerza bruta.

### **Aarón:**

Aporte en lo siguiente:

- Las plantillas de las clases.
- La creación total de la clase jugador, Luigui, además de mejorar las clases Mesa y Carta, por último en empezar a desarrollar la clase GamePlay (La interfaz del juego en sí).
- Algunos detalles visuales de SFML
- Hice cambios en el flujo del juego, el cómo se ejecutan y cómo se llaman las funciones en general.
- Separar el proyecto en librerías.

La parte más importante fueron los retos técnicos, quitando algunos detalles que tenían que ver con SFML que solamente era investigar cómo se hace cierta función. En realidad lo difícil fue la lógica, me enfrente principalmente al error de los índices no válidos en los vectores tipo Carta, también que muchas veces mi flujo de lógica estaba mal.

Yo puedo desglosar los problemas que me enfrente en tres tipos: Crasheos, errores de sintaxis y errores de lógica. Siempre que se ejecutaba el programa y salía una pantalla que marca error en algo sabía que era por un crasheo. Para los errores de lógica me daba cuenta que lo que había programado no se veía reflejado tal como lo planeado en el programa y para los de sintaxis el compilador me indicaba en donde estaba mal.

Especialmente con errores de lógica lo que hace es imprimir cout en la pantalla para validar que acción se estaba haciendo y con base a eso modificar, otras veces rediseñar el flujo por completo de aquella función o llamadas para ver si ya estaba bien. Para los errores de crasheo no tenía ni idea que significa el código de error que me lanzaba, así que le pedía a Gemini que dijera ese error, al final, resultó que todos los Crasheos eran porque como elimina un objeto Carta de un vector, ese índice quedaba no válido y crasheaba, para solucionarlo tuve que en muchos casos crear copias del objeto para retornarlos y así eliminar el objeto viejo, otras veces era cambiar la estructura del for o guardas sus índices para eliminarlos más al rato.

Pienso que en este código desarrolle principalmente la paciencia soportar errores, el temple para revisar y replantear las ideas que tengo y cómo aplicarlas, la comunicación de avisar que hice a mis compañeros y comentar qué decisiones sugiero tomar pero sobretodo, el pensar ordenadamente es algo que desarrolle bastante con este proyecto.

### **ICC\_1:**

Para identificar los problemas depende mucho del tipo que eran. Principalmente para problemas de lógica, la solución era cuestionar y reformular la idea que no se estaba aplicando. También para los problemas de crasheo se pidió a un agente de inteligencia artificial que explicara qué significa este crasheo.

Tuvimos que abandonar la idea del multijugador por complejidad y por cuestiones de tiempo, también la idea de tener distintas dificultades para las máquinas por cuestiones de tiempo.

Principalmente para resolver problemas con orientado a objetos hicimos uso de las validaciones, por medio de función privadas aquí aplicamos encapsulamiento porque el usuario no tiene que acceder a esas funciones. También abstracción porque representamos lo fundamental en las clases y hacemos que el usuario solo pueda ver casi nada del proceso

interno y complejidad de la lógica. Otro concepto es el poliformismo, nos simplificó bastante las líneas de código, permite visualizar y encontrar un error que surgiera de forma más rápida.

Consideramos que en muchos casos, sobre todo cuando hablamos de implementar nuevas funciones y lógica, tuvimos que pensar y plantear de qué forma impacta esta nueva incorporación. También que pasos ordenados debería de tener la nueva función o flujo lógico para mejorar una tecnología ya prediseñada por nosotros y hacer el proceso de innovación..

### **Anexos:**

Para poder hacer la función de mezclar cartas:

<https://learn.microsoft.com/es-es/cpp/standard-library/random?view=msvc-170>

Se utilizó la librería <algorithm> para ordenar algunas funciones.

<https://learn.microsoft.com/es-es/cpp/standard-library/algorithm?view=msvc-170>

Si usted programador, desea usar la librería gráfica utilizada en nuestro proyecto:

<https://www.sfml-dev.org/download/sfml/3.0.2/>